

TypeScript - Principiante

Tema 18: Promises y async/await

20 ejercicios progresivos, sin soluciones, para practicar Promise, resolve, reject, then, catch, finally, async, await, manejo de errores y operaciones asincronas tipadas.

Ejercicio 1 - Promise resuelta

Crea una Promise que se resuelva con el mensaje Operacion completada.

Requisitos

- Tipa explicitamente la Promise.
- Consume el resultado con then.

Ejemplo de ejecucion

Operacion completada

Objetivo

Introducir Promise y resolve.

Pista

El tipo generico indica el valor que entrega la Promise.

Ejercicio 2 - Promise numerica

Crea una Promise que devuelva un numero.

Requisitos

- Usa resolve.
- Multiplica el resultado por dos en then.

Ejemplo de ejecucion

Resultado: 20

Objetivo

Practicar Promises tipadas.

Pista

El valor recibido en then conserva el tipo de la Promise.

Ejercicio 3 - Promise rechazada

Crea una Promise que pueda ser rechazada y maneja el error con catch.

Requisitos

- Usa reject.
- Muestra un mensaje de error.

Ejemplo de ejecucion

Error: operacion fallida

Objetivo

Practicar rechazo de Promises.

Pista

catch permite manejar una Promise rechazada.

Ejercicio 4 - finally

Crea una Promise y agrega then, catch y finally.

Requisitos

- finally debe ejecutarse independientemente del resultado.
- Muestra un mensaje al finalizar.

Ejemplo de ejecucion

Proceso finalizado

Objetivo

Practicar finally.

Pista

finally es util para tareas de limpieza o finalizacion.

Ejercicio 5 - Simular retraso

Crea una funcion delay que devuelva Promise y se resuelva despues de un tiempo.

Requisitos

- Usa setTimeout dentro de la Promise.
- El tiempo se recibe como parametro.

Ejemplo de ejecucion

Esperando...

Terminado

Objetivo

Comprender operaciones asincronas simuladas.

Pista

Envuelve setTimeout dentro de una Promise.

Ejercicio 6 - Funcion async

Crea una funcion async que devuelva un saludo.

Requisitos

- Declara correctamente el tipo de retorno.
- Consume la funcion con await o then.

Ejemplo de ejecucion

Hola, TypeScript

Objetivo

Introducir funciones async.

Pista

Una funcion async siempre devuelve una Promise.

Ejercicio 7 - Await basico

Crea una funcion getNumber que devuelva Promise y otra funcion async que espere el resultado.

Requisitos

- Usa await.
- Realiza un calculo despues de esperar.

Ejemplo de ejecucion

Resultado: 30

Objetivo

Practicar await.

Pista

await obtiene el valor resuelto de una Promise.

Ejercicio 8 - Usuario asincrono

Define User y crea getUser(id) que devuelva Promise.

Requisitos

- Simula la consulta.
- Consume el usuario con await.
- No uses any.

Ejemplo de ejecucion

Usuario: Ana

Objetivo

Combinar Promises con objetos tipados.

Pista

Promise conserva la estructura del usuario.

Ejercicio 9 - Usuario inexistente

Modifica getUser para rechazar la Promise cuando el id no exista.

Requisitos

- Maneja el error con try/catch.
- No ignores el error.

Ejemplo de ejecucion

Usuario no encontrado

Objetivo

Practicar errores con async/await.

Pista

try/catch puede envolver operaciones await.

Ejercicio 10 - Lista asincrona

Crea getProducts que devuelva Promise.

Requisitos

- Define Product.
- Obtiene y recorre la lista usando await.

Ejemplo de ejecucion

Productos encontrados: 5

Objetivo

Practicar colecciones asincronas tipadas.

Pista

El resultado esperado puede ser un array tipado.

Ejercicio 11 - Crear registro

Crea createUser que reciba datos de usuario y devuelva Promise.

Requisitos

- Simula la generacion del id.
- Valida datos antes de resolver.

Ejemplo de ejecucion

Usuario creado: Carlos

Objetivo

Modelar operaciones asincronas de escritura.

Pista

La Promise puede resolver con la entidad creada.

Ejercicio 12 - Secuencia de operaciones

Crea getUser y getOrdersByUser. Primero obtiene un usuario y despues sus pedidos.

Requisitos

- La segunda operacion depende de la primera.
- Usa async/await.

Ejemplo de ejecucion

Ana - Pedidos: 3

Objetivo

Practicar operaciones asincronas dependientes.

Pista

Espera el primer resultado antes de iniciar la operacion que lo necesita.

Ejercicio 13 - Operaciones paralelas

Crea getUsers y getProducts independientes y ejecutalas en paralelo.

Requisitos

- Usa Promise.all.
- Tipa correctamente ambos resultados.

Ejemplo de ejecucion

Usuarios: 4 | Productos: 8

Objetivo

Practicar concurrencia con Promise.all.

Pista

Si las operaciones no dependen entre si, no siempre necesitas esperarlas una por una.

Ejercicio 14 - Varias consultas

Simula tres funciones asincronas para obtener perfil, permisos y configuracion de un usuario.

Requisitos

- Ejecuta las consultas independientes en paralelo.
- Combina los resultados.

Ejemplo de ejecucion

Perfil cargado

Permisos cargados

Configuracion cargada

Objetivo

Combinar multiples Promises.

Pista

Promise.all espera que todas se completen correctamente.

Ejercicio 15 - Resultado parcial

Crea varias Promises donde alguna pueda fallar y procesa todos los resultados.

Requisitos

- Investiga Promise.allSettled.
- Distingue fulfilled y rejected.
- Tipa los valores.

Ejemplo de ejecucion

Exitosas: 2 | Fallidas: 1

Objetivo

Practicar operaciones donde un fallo no debe descartar todos los resultados.

Pista

allSettled conserva el resultado individual de cada Promise.

Ejercicio 16 - Timeout conceptual

Crea una operacion asincrona y una Promise que falle despues de cierto tiempo. Compite ambas.

Requisitos

- Usa Promise.race.
- Maneja el caso de timeout.

Ejemplo de ejecucion

Error: tiempo de espera agotado

Objetivo

Practicar competencia entre Promises.

Pista

race se resuelve o rechaza con la primera Promise que termine.

Ejercicio 17 - Servicio asincrono

Define IUserRepository con getById y getAll que devuelvan Promises.

Requisitos

- Crea una implementacion en memoria.
- Crea UserService que use el repositorio.
- Todos los metodos asincronos deben estar tipados.

Ejemplo de ejecucion

Usuarios cargados: 5

Objetivo

Aplicar asincronia a repositorios y servicios.

Pista

Los contratos tambien deben expresar Promise en sus retornos.

Ejercicio 18 - Manejo de unknown en catch

Crea una operacion async que pueda lanzar diferentes errores.

Requisitos

- Trata el valor capturado de forma segura.
- Usa instanceof Error cuando corresponda.
- No uses any.

Ejemplo de ejecucion

Error controlado: conexion fallida

Objetivo

Combinar asincronia con manejo seguro de errores.

Pista

No asumas que todo valor capturado tiene message.

Ejercicio 19 - Flujo de pedido

Simula createOrder, reserveStock y processPayment como funciones asincronas.

Requisitos

- Ejecuta las operaciones en el orden correcto.
- Si falla una, detiene el flujo.
- Usa try/catch.
- Devuelve un resultado tipado.

Ejemplo de ejecucion

Pedido creado

Stock reservado

Pago procesado

Objetivo

Modelar un flujo asincrono dependiente.

Pista

Cada paso puede necesitar el resultado del anterior.

Ejercicio 20 - Sistema asincrono de gastos

Construye un pequeño sistema con ExpenseRepository y ExpenseService cuyos metodos sean asincronos.

Requisitos

- Define Expense.
- getAll devuelve Promise.
- getByid devuelve Promise.
- add devuelve Promise.
- ExpenseService permite registrar y totalizar gastos.
- Simula latencia.
- Maneja errores con try/catch.
- Ejecuta consultas independientes en paralelo cuando tenga sentido.
- No uses any.

Ejemplo de ejecucion

Gasto registrado

Gastos cargados: 5

Total: \$3250

Objetivo

Integrar Promises, async/await, tipos, repositorios y servicios.

Pista

Mantiene la misma separacion de responsabilidades aunque las operaciones sean asincronas.